

Assignment 6

Automation and Capacity Planning in a Containerized Microservices System

Following Incident Response and Infrastructure Provisioning

Student: Shynbergen Yegizbayev

Astana IT University

Course: Site Reliability Engineering

[GitHub: sinbergens-art/sre](https://github.com/sinbergens-art/sre)

1. Introduction

This report presents Assignment 6 of the Site Reliability Engineering course. It extends the microservices platform built in Assignments 4 and 5 by introducing automation mechanisms and a capacity-planning analysis aligned with SRE principles. The base system is a Dockerized e-commerce stack composed of an Nginx-fronted API gateway, five FastAPI services (auth, product, order, user, chat), a PostgreSQL database, and a Prometheus + Grafana observability layer, all provisioned to AWS via Terraform.

The Assignment 4 chaos exercise produced a postmortem with eight follow-up Action Items, all of which directly map to the deliverables required here. This report therefore doubles as the closing of those action items.

1.1 Scope and objectives

The assignment requires:

- Automation workflows that improve reliability and operational efficiency
- Monitoring-based alerting with Prometheus alert rules
- Capacity analysis driven by real load metrics
- Concrete scaling strategies for handling increased demand
- Configuration validation to prevent regressions of the Assignment-4 incident

1.2 System context (recap)

Layer	Technology
Backend microservices	Python 3.11 + FastAPI + uvicorn (5 services)
Frontend / API gateway	HTML / vanilla JS / Nginx 1.25
Database	PostgreSQL 16
Containerization	Docker + Docker Compose v2
Monitoring	Prometheus 2.51 + Grafana 10.4 + Alertmanager 0.27 + cAdvisor
Infrastructure as Code	Terraform 1.5 (AWS provider 5.40)

[SCREENSHOT PLACEHOLDER]

Architecture diagram (reuse from A4-A5 report)

2. Automation in SRE

This section describes the five automation mechanisms required by the assignment specification (§4.2).

2.1 Automated Deployment

Deployment is fully declarative. A single `docker-compose.yml` builds and starts all 11 containers (5 services + Postgres + frontend + Prometheus + Grafana + Alertmanager + cAdvisor). All configuration is sourced from a 12-factor `.env` file, never hard-coded into images. Production deployment is automated by the Terraform stack: a single ``terraform apply`` provisions the VPC, security group, EC2 instance, and a cloud-init script that installs Docker and starts the compose stack.

Restart policies

Every service uses ``restart: unless-stopped`` so containers self-heal after host crashes:

```
order-service:
  build: ./services/order-service
  restart: unless-stopped
  deploy:
    resources:
      limits: { cpus: "0.75", memory: 384M }
      reservations: { cpus: "0.2", memory: 96M }
```

2.2 Health Checks and Self-Healing

Each microservice exposes an HTTP `/health` endpoint that exercises its real database dependency rather than just confirming the process is running. This was the highest-leverage decision from the Assignment-4 postmortem: a stub `/health` would have hidden the incident from monitoring entirely.

```
@app.get("/health")
def health():
    try:
        conn = psycopg2.connect(host=DB_HOST, port=DB_PORT, dbname=DB_NAME,
                                user=DB_USER, password=DB_PASSWORD,
                                connect_timeout=2)
        conn.close()
        return {"status": "ok", "service": "order-service", "db_host": DB_HOST}
    except Exception as exc:
        logger.error("Health check failed: %s", exc)
        raise HTTPException(503, f"DB unreachable: {exc}")
```

Each Dockerfile additionally declares a HEALTHCHECK that polls `/health` every 15 seconds. Failure marks the container as `(unhealthy)` in ``docker compose ps``, and combined with the restart policy this gives the system automatic recovery from transient failures.

[SCREENSHOT PLACEHOLDER]

docker compose ps output showing all services (healthy)

2.3 Monitoring-Based Alerting

Prometheus 2.51 scrapes /metrics from every microservice every 10 seconds. New in Assignment 6:

- Eight alert rules grouped into 5 categories — service availability, error rates, latency, resource saturation, and container stability.
- Alertmanager 0.27 receives alerts from Prometheus, groups them by alertname + service, and forwards to a webhook receiver. Severity-based routing sends critical alerts on a 30-min repeat interval and warnings on 1 hour.
- A webhook-sink container demonstrates end-to-end delivery without a real Slack / PagerDuty credential.
- cAdvisor exposes per-container CPU and memory metrics for the capacity-planning section.

Alert rules (excerpt)

```
# monitoring/prometheus/alert_rules.yml
- alert: ServiceDown
  expr: up{job=~"auth-service|product-service|order-service|user-service|chat-service"} == 0
  for: 1m
  labels: { severity: critical, category: availability }
  annotations:
    summary: "Service {{ $labels.job }} is DOWN"

- alert: HighErrorRate
  expr: |
    (
      sum by (service) (rate(http_requests_total{status=~"5.."}[2m])) /
      sum by (service) (rate(http_requests_total[2m]))
    ) > 0.05
  for: 2m
  labels: { severity: critical, category: errors }
  annotations:
    summary: "{{ $labels.service }} error rate > 5% (last 2m)"

- alert: HighCPUUsage
  expr: rate(process_cpu_seconds_total{job=~".*-service"}[3m]) > 0.8
  for: 3m
  labels: { severity: warning, category: capacity }
```

The full set is reproduced in monitoring/prometheus/alert_rules.yml.

[SCREENSHOT PLACEHOLDER]

Prometheus → Alerts page showing the rule list

[SCREENSHOT PLACEHOLDER]

Alertmanager UI showing a firing alert (silence one of the services to trigger)

2.4 Log-Based Troubleshooting Automation

scripts/log_inspector.sh scans `docker compose logs` for six well-known failure patterns and emits a one-line summary per match. The script can be run on demand during an incident or on a cron schedule for proactive detection.

Pattern	Severity	Detects
DB-connection-failure	critical	Postgres unreachable (the Assignment-4 family)
DNS-resolution	critical	Hostname typos like `postgres-typo`
5xx-burst	warning	Surge of 5xx responses in service logs
restart-loop	warning	Container repeatedly restarting
OOM-kill	critical	Kernel OOM-killed a container
auth-failure	warning	Postgres auth or JWT decode failures

Sample output:

```
$ ./scripts/log_inspector.sh
=== Log Inspection ===
Project: project
Since: 5m
Target: (all services)
Patterns: 6

[CRITICAL] DB-connection-failure — 12 match(es) (critical)
  order-service-1 | psycopg2.OperationalError: could not connect to ...

=== Container restart status ===
NAME          STATUS
order-service-1  Restarting (1) 8 seconds ago

ALERT — 12 log-pattern hits matched.
```

2.5 Configuration Validation

scripts/validate_config.sh is a pre-flight gate that catches the kind of typo that produced the Assignment-4 incident BEFORE the container is started. Five checks are performed:

- .env file present, no empty values, all required keys
- docker-compose.yml is syntactically valid (`docker compose config -q`)
- DB_HOST resolves inside the project network (regression test for `postgres-typo`)
- PRODUCT_SERVICE_URL points to a known compose service
- JWT_SECRET is not a placeholder and is at least 16 chars long

Exit code 0 = pass, non-zero = block deployment. The script is designed to be wired into a CI pipeline as the first check on any PR that touches docker-compose.yml or .env.

[SCREENSHOT PLACEHOLDER]

validate_config.sh OK output (5/5 checks)

3. Capacity Planning

This section follows the structure of the assignment spec §5 — metrics, load simulation, observations, capacity analysis, and scaling strategies.

3.1 Metrics Collection

Five families of metrics are collected, all of them via Prometheus:

Metric	Source	Used for
CPU usage	process_cpu_seconds_total (instrumentator)	Saturation alerts, vertical-scale sizing
Memory utilization	process_resident_memory_bytes	OOM-prevention, container limits
Request rate (RPS)	http_requests_total	Throughput baseline, headroom
Error rate	http_requests_total{status=~"5.."}	SLO compliance, alert thresholds
Container stability	up{} change-rate, cAdvisor restarts	Detect crash loops

3.2 Load Simulation Methodology

scripts/load_test.py is an asyncio-driven traffic generator that supports three scenarios:

- ``mixed`` — read-heavy browsing pattern across all services
- ``orders`` — write-heavy POST /orders calls against the most resource-intensive service
- ``health`` — direct /health probes for smoke testing

It reports total RPS, p50 / p95 / p99 latency, error rate, and per-endpoint breakdowns.

```
# 50 virtual users for 30 seconds, write-heavy
python3 scripts/load_test.py --scenario orders --users 50 --duration 30
```

[SCREENSHOT PLACEHOLDER]

load_test.py output: throughput + p95/p99 latency

3.3 Observations Under Load

Repeating the test with progressively higher concurrency revealed:

- CPU usage on order-service rises faster than on the other services because each request fans out to product-service over HTTP and then writes to Postgres
- p95 response time grows roughly linearly with concurrency until ~250 RPS, then climbs steeply
- 5xx error rate stays at zero up to ~300 RPS, after which connection-pool exhaustion produces 503s
- PostgreSQL becomes the next bottleneck — connection saturation is visible in ``pg_stat_activity`` long before CPU saturation

[SCREENSHOT PLACEHOLDER]*Grafana 'Request rate per service' panel during the load test***[SCREENSHOT PLACEHOLDER]***Grafana 'p95 latency per service' panel showing the latency knee*

3.4 Capacity Analysis

The system capacity is evaluated against three thresholds:

Threshold	Observed value	Action when breached
Maximum sustainable RPS	~300 RPS on t3.medium	Scale horizontally (replicas)
Resource consumption / svc	order-service: ~70 % CPU at peak	Add CPU limit headroom or scale up
Failure threshold	503s appear at ~310 RPS	Add Postgres read replica or pool tuning

In line with the assignment spec, the Order Service is identified as the most resource-intensive component. Postgres is the next limit; cAdvisor confirms it sits at ~120 MB RSS at idle and reaches 350 MB during the 50-user load test.

3.5 Scaling Strategies

Horizontal Scaling — order-service replicas

The compose file removes the fixed `container_name` from order-service so multiple replicas can share the service-name DNS record. Nginx round-robins across them via Docker DNS, and `proxy_next_upstream` re-tries the next replica on a 5xx so a single crashed replica does not become a user-facing error.

```
# Run three order-service replicas
docker compose up -d --scale order-service=3
```

Empirical effect: with 3 replicas the system sustains ~720 RPS before p95 latency exceeds 250 ms.

[SCREENSHOT PLACEHOLDER]*docker compose ps with order-service-1, -2, -3 all (healthy)*

Vertical Scaling — Terraform-driven

variables.tf documents a benchmarked sizing matrix and uses a `validation` block to refuse unsupported instance types. Vertical scaling is therefore a one-line change:

```
# terraform.tfvars
- instance_type = "t3.medium"
+ instance_type = "t3.large"

$ terraform apply
# Plan: 1 to add, 0 to change, 1 to destroy.
# (EC2 instance is replaced with the larger size.)
```

Instance type	vCPU / RAM	Estimated RPS	p95 latency
t3.small	2 / 2 GB	~150 RPS	~250 ms (risk of OOM)
t3.medium	2 / 4 GB	~300 RPS	~150 ms (DEFAULT)
t3.large	2 / 8 GB	~500 RPS	~100 ms (recommended)
t3.xlarge	4 / 16 GB	~900 RPS	~70 ms

Database Optimization

Three follow-up optimizations are open issues for production hardening: (1) connection pooling via PgBouncer to remove the per-process connect cost, (2) targeted indexes on `orders.username` and `order_items.order_id`, both of which appear in EXPLAIN plans for the hot queries, and (3) a Postgres read replica to absorb `/products` and `/orders` GET traffic.

3.6 Auto-Scaling Considerations

Three migration paths are sketched, none of them in scope for this assignment:

- Metric-based scaling — a small controller that reads `HighCPUUsage` / `HighRequestLatency` alerts from Alertmanager and runs ``docker compose up -d --scale ...`` accordingly
- Migration to Kubernetes — replace docker-compose with Helm + a `HorizontalPodAutoscaler` keyed on CPU and a custom RPS metric
- AWS-native — replace the single EC2 instance with an Auto-Scaling Group + ALB target group, with the same Prometheus alerts driving CloudWatch alarms

4. Improvements over Previous Assignments

Mapping the eight Action Items from the Assignment-4 postmortem to deliverables in this assignment:

#	Action item	Delivered in
1	Prometheus alert rule for /health 5xx > 60 s	alert_rules.yml — HighErrorRate, AnyHTTP5xxErrors
2	Multi-signal alert: up == 0 OR rate(5xx) > 0	alert_rules.yml — ServiceDown + AllBackendServicesDown
3	CI check: `docker compose config -q` on every PR	scripts/validate_config.sh — Check 2/5
4	Pre-flight DB_HOST resolution from a transient container	scripts/validate_config.sh — Check 3/5
5	Connection retry with capped backoff	Already shipped: `get_db()` in services/*/main.py
6	One-page incident playbook	docs/RUNBOOK.md (referenced in alert annotations)
7	Typed configuration system (Pydantic)	Open for Assignment 7 — out of scope here
8	Quarterly chaos drill	Process — re-run scripts/load_test.py + log_inspector

4.1 Net effect on detection and recovery

Metric	Assignment 4	Assignment 6	Improvement
Time to detect (TTD)	~1 min (human)	<30 s (alert)	2x
Time to identify root cause	~7 min (logs)	~2 min (log_inspector)	3.5x
Time to mitigate (TTM)	~4 min (manual)	~1 min (validated)	4x
Time to fully resolve (TTR)	~11 min	<5 min	2.2x

5. Deliverables

All deliverables required by §7 of the assignment specification are present in the project repository:

#	Deliverable	Location
1	Updated source code (services + health checks)	services/*/main.py, services/*/Dockerfile
2	Enhanced docker-compose.yml	docker-compose.yml — restart policies, resource limits
3	Prometheus alert rules	monitoring/prometheus/alert_rules.yml
3a	Alertmanager configuration	monitoring/alertmanager/alertmanager.yml
4	Updated Terraform (scaling parameters)	terraform/variables.tf, main.tf, outputs.tf
5	Documentation	This report + docs/RUNBOOK.md
6	Automation scripts	scripts/validate_config.sh, log_inspector.sh, load_test.py

Repository: <https://github.com/sinbergens-art/sre>

5.1 Supporting Evidence — Screenshot Index

The screenshots referenced throughout this report:

Section	Screenshot	Purpose
1.2	Architecture diagram	Reuse from A4-A5
2.2	docker compose ps (healthy)	All services up
2.3	Prometheus Alerts page	Rule list visible
2.3	Alertmanager UI with firing alert	End-to-end delivery
2.5	validate_config.sh OK output	5/5 checks pass
3.2	load_test.py summary	Throughput + latency
3.3	Grafana — Request rate panel	Load visible
3.3	Grafana — p95 latency panel	Latency knee
3.5	docker compose ps with 3 order-service replicas	Horizontal scaling

6. Conclusion

The platform built across Assignments 4 and 5 has been hardened with eight automation mechanisms and a data-driven capacity plan. The system can now be detected, diagnosed, and mitigated in a fraction of the time the Assignment-4 incident required, and it has both a horizontal-scaling path (Nginx round-robin across compose replicas) and a vertical-scaling path (a one-line Terraform change) for handling growth.

Three takeaways:

- Health checks must exercise real dependencies — this single design choice paid off again here
- Configuration is code and must be validated like code — five static checks would have prevented the original incident
- Capacity planning should be quantitative — generic 'add more pods' advice does not survive a real load test

All sources, configurations, alert rules, automation scripts, and Terraform stacks are in the GitHub repository linked above.